

PostgreSQL SQL Dashboard

Mevcut Veri Kullanımı ve İterasyon 1.1

Geliştirme Değerlendirme Raporu

Bu rapor, mevcut PostgreSQL ve PoWA altyapısından gelen metriklerin dashboard'da ne ölçüde kullanıldığını, kullanılmayan yüksek değerli alanları ve ikinci iterasyona geçmeden önce önerilen İterasyon 1.1 geliştirmelerini açıklar.

İncelenen proje: github.com/McanKul/sql-dashboard

Rapor tarihi: 23 Temmuz 2026

Durum: Teknik değerlendirme ve geliştirme önerisi

Hedef okuyucu: Backend, veritabanı ve ürün geliştirme ekibi

İçindekiler

1	Yönetici Özeti	2
1.1	Ana değerlendirme	2
2	Raporun Amacı ve Kapsamı	2
2.1	Kapsam içi	2
2.2	Kapsam dışı	3
3	Mevcut Mimari ve Veri Akışı	3
3.1	Bileşen sorumlulukları	3
4	Mevcut Veri Kaynakları	4
4.1	pg_stat_statements	4
4.1.1	Kimlik ve sorgu alanları	4
4.1.2	Çalıştırma alanları	4
4.1.3	Cache, I/O ve temp alanları	4
4.1.4	WAL, JIT ve paralellik alanları	5
4.1.5	Planning alanları	5
4.2	pg_stat_statements_info	5
4.3	Tablo ve index istatistikleri	5
4.4	Anlık aktivite ve lock verileri	5
5	Dashboard'un Bugünkü Kullanım Alanı	5
5.1	Bugünkü kullanımın güçlü tarafı	6
5.2	Bugünkü kullanımın sınırı	6
6	Kullanılmayan Yüksek Değerli Analizler	6
6.1	Performans kararlılığı	6
6.1.1	Önerilen türetilmiş metrikler	6
6.2	Satır verimliliği	7
6.3	Cache ve I/O bağımlılığı	7
6.4	Temp spill analizi	7
6.5	WAL ve yazma etkisi	8
6.6	Planning maliyeti	8
6.7	Parallel worker ve JIT görünürlüğü	8
7	PoWA Geçmişinden Çıkarılabilecek Ek Değer	9
7.1	Top movers	9
7.2	Yeni ağır sorgular	9
7.3	Saatlik ve günlük yük haritası	9
7.4	Deployment öncesi ve sonrası	9
8	Veri Kullanım Matrisi	9
9	Önerilen İterasyon 1.1	10
9.1	P0 - Ürün değerini doğrudan artıran işler	10
9.2	P1 - Analiz derinliğini artıran işler	11
9.3	P2 - İleri seviye detaylar	11
10	Önerilen Dashboard Tasarımı	11
10.1	Sorgu listesi sade kalmalıdır	11
10.2	Sorgu detay sekmeleri	11

11 Önerilen Finding Kuralları	12
12 Veri Kalitesi ve Yanlış Pozitif Riskleri	12
12.1 Sayaç resetleri	12
12.2 Deallocation	12
12.3 Düşük örnek sayısı	13
12.4 Cache yorumları	13
12.5 Satır alanının anlamı	13
12.6 Query ID kararlılığı	13
13 Mevcut Veriyle Üretilmeyecek Sonuçlar	13
14 Uygulama Planı	13
14.1 Aşama 1 - Veri uygunluk kontrolü	13
14.2 Aşama 2 - Repository ve SQL katmanı	14
14.3 Aşama 3 - FastAPI	14
14.4 Aşama 4 - React dashboard	14
14.5 Aşama 5 - Test ve doğrulama	14
15 Kabul Kriterleri	15
16 Beklenen Ürün Etkisi	15
17 Sonuç	15
18 Kaynaklar	17

1 Yönetici Özeti

Yönetici Özeti

Mevcut ilk iterasyon başarısız veya yetersiz değildir. Sistem bilinçli olarak en temel ürün sorusuna odaklanmıştır: **“Önce hangi sorguya bakmalıyız?”** Bununla birlikte, `pg_stat_statements`, PostgreSQL istatistik görünümü ve PoWA geçmiş verisi, bugün dashboard’da kullanılan alandan belirgin şekilde daha geniş bir veri seti sağlamaktadır. En büyük fırsat daha fazla ham kolon göstermek değil; mevcut ham veriyi **kararlılık, I/O bağımlılığı, temp spill, satır verimliliği, WAL etkisi, veri kalitesi ve tarihsel değişim** gibi anlamlı analizlere dönüştürmektir.

Önerilen yaklaşım, index veya SQL rewrite önerilerine geçmeden önce küçük bir **İterasyon 1.1 - Gözlemlenebilirlik Zenginleştirme** çalışması yapmaktır. Bu çalışma çoğunlukla yeni extension gerektirmez ve mevcut mimariyi değiştirmeden dashboard’un değerini önemli ölçüde artırır.

1.1 Ana değerlendirme

- Veri toplama kapasitesi, mevcut dashboard kullanımından daha geniştir.
- İlk iterasyonda kullanılan veri seti doğru bir minimum çekirdektir.
- Mevcut kaynaklardan yeni extension kurmadan daha fazla analiz üretilebilir.
- Öncelik, ana tabloya daha fazla kolon eklemek değil, sorgu detayını zenginleştirmek olmalıdır.
- Gerçek p95/p99, execution plan düğümleri ve güvenilir index önerileri mevcut veriyle üretilemez; bunlar sonraki iterasyonların konusudur.

Önerilen karar

İkinci iterasyondaki index ve JOIN önerilerine başlamadan önce, 2-3 haftalık sınırlı kapsamlı bir İterasyon 1.1 uygulanmalıdır. Bu iterasyon; cache/I/O, çalışma süresi değişkenliği, temp spill, satır verimliliği, WAL yükü, top movers ve veri kalitesi özelliklerini kapsamalıdır.

2 Raporun Amacı ve Kapsamı

Bu rapor aşağıdaki sorulara cevap vermek amacıyla hazırlanmıştır:

1. Mevcut mimaride hangi bileşen hangi veriyi sağlamaktadır?
2. Dashboard bugün bu verilerin hangi bölümünü kullanmaktadır?
3. Hazır geldiği halde henüz kullanılmayan değerli metrikler nelerdir?
4. Bu metriklerden hangi güvenilir ürün özellikleri üretilebilir?
5. Hangi çıktılar mevcut veriyle üretilemez veya yanıltıcı olur?
6. İterasyon 1.1 için önerilen uygulama ve kabul planı nedir?

2.1 Kapsam içi

- `pg_stat_statements` sorgu istatistikleri
- `pg_stat_statements_info` veri kalitesi bilgileri
- PostgreSQL tablo, index ve I/O istatistikleri

- PoWA Collector ve PoWA Repository tarihsel veri rolü
- Mevcut Impact Score ve dönem karşılaştırması yaklaşımı
- Yeni extension olmadan üretilebilecek dashboard geliştirmeleri

2.2 Kapsam dışı

Bu raporda çözülmeyen alanlar

Execution plan toplama ve PEV2 entegrasyonu, otomatik CREATE/DROP INDEX önerileri, pg_qualstats, HypoPG, gereksiz JOIN tespiti, gerçek çağrı bazlı percentile ölçümü ve otomatik SQL rewrite ikinci iterasyon veya sonrasının konusudur.

3 Mevcut Mimari ve Veri Akışı

Mevcut mimarinin temel veri akışı aşağıdaki gibidir:

```
Source PostgreSQL :5432
|-- pg_stat_statements
|-- table/index statistics
|-- optional current activity statistics
    |
    v
PoWA Collector
    |
    v
PoWA Repository PostgreSQL :5433
|-- historical snapshots
|-- retention
|-- normalized data access
    |
    v
Advisor SQL Layer
|-- period deltas
|-- regression
|-- Impact Score
|-- priority and findings
    |
    v
FastAPI
    |
    v
React Dashboard
```

3.1 Bileşen sorumlulukları

Bileşen	Temel sorumluluk	Sağladığı değer
pg_stat_statements	Normalize sorgu kalıpları için kümülatif planlama ve çalışma istatistikleri toplamak	Çağrı, süre, blok, temp, WAL, JIT ve paralellik gibi sorgu bazlı ham metrikler
PostgreSQL istatistik görünümüleri	Tablo, index, I/O, session ve lock durumunu raporlamak	Sorgu dışındaki veritabanı nesnesi ve anlık sağlık sinyalleri

Bileşen	Temel sorumluluk	Sağladığı değer
PoWA Collector	Kaynak sunuculardan periyodik snapshot almak	Otomatik ve çoklu kaynaklı veri toplama
PoWA Repository	Metrikleri tarihsel olarak saklamak ve ortak görünüm sağlamak	1/7/30 günlük dönem, delta ve trend hesaplama temeli
Advisor SQL katmanı	Ham veriyi karar metriklerine çevirmek	Regresyon, yük payı, Impact Score, priority ve findings
FastAPI	Analiz sonuçlarını güvenli API olarak sunmak	Filtreleme, sayfalama, maskeleme, export ve annotation
React dashboard	Sonuçları kullanıcıya açıklanabilir şekilde göstermek	Önceliklendirme, detay ve sağlık ekranları

4 Mevcut Veri Kaynakları

4.1 pg_stat_statements

PostgreSQL'in resmi `pg_stat_statements` modülü, sunucuda çalıştırılan SQL ifadelerinin planlama ve execution istatistiklerini takip eder. Her kayıt; kullanıcı, veritabanı, normalize sorgu kimliği ve top-level durumu gibi alanların kombinasyonuna göre tutulur. Sabit değerler çoğunlukla \$1, \$2 biçiminde normalize edilir.

4.1.1 Kimlik ve sorgu alanları

- `userid`, `dbid`, `toplevel`
- `queryid`
- `query`

4.1.2 Çalıştırma alanları

- `calls`, `rows`
- `total_exec_time`, `mean_exec_time`
- `min_exec_time`, `max_exec_time`, `stddev_exec_time`

4.1.3 Cache, I/O ve temp alanları

- `shared_blks_hit`, `shared_blks_read`
- `shared_blks_dirtied`, `shared_blks_written`
- `temp_blks_read`, `temp_blks_written`
- `shared_blk_read_time`, `shared_blk_write_time`
- `temp_blk_read_time`, `temp_blk_write_time`

I/O süre alanları yalnızca `track_io_timing` etkin olduğunda anlamlıdır.

4.1.4 WAL, JIT ve paralellik alanları

- wal_records, wal_fpi, wal_bytes, wal_buffers_full
- jit_functions ve JIT süreleri
- parallel_workers_to_launch, parallel_workers_launched

4.1.5 Planning alanları

- plans
- total_plan_time, mean_plan_time
- min_plan_time, max_plan_time, stddev_plan_time

Bu alanlar `pg_stat_statements.track_planning` etkin değilse sıfır olur. Planning takibi ek maliyet oluşturabileceği için ayrı değerlendirilmelidir.

4.2 pg_stat_statements_info

Bu görünüm veri kalitesi açısından iki önemli alan sağlar:

- stats_reset: sayaçların son sıfırlanma zamanı
- dealloc: takip kapasitesi aşıldığında çıkarılan düşük kullanımlı kayıt sayısı

Bu iki alan, dashboard sonuçlarının ne kadar güvenilir olduğunu kullanıcıya anlatmak için kullanılmalıdır.

4.3 Tablo ve index istatistikleri

- pg_stat_user_tables: sequential scan, index scan, live/dead tuple, DML ve vacuum/analyze bilgileri
- pg_stat_user_indexes: index kullanım sayısı, okunan ve getirilen tuple bilgileri
- pg_statio_user_tables: heap, index ve TOAST blok okuma/hit bilgileri
- pg_statio_user_indexes: index bazlı I/O bilgileri

Bu sayaçların çoğu kümülatiftir. Zaman aralığına ait değişimi göstermek için iki snapshot arasındaki delta hesaplanmalıdır.

4.4 Anlık aktivite ve lock verileri

- pg_stat_activity: aktif sorgular, session durumları, transaction başlangıcı ve wait event bilgileri
- pg_locks: mevcut lock'lar ve blocker/waiter ilişkileri

Bunlar tarihsel sorgu yükünden farklıdır. `pg_stat_statements` “geçmişte toplam ne oldu?” sorusunu, `pg_stat_activity` ise “şu anda ne oluyor?” sorusunu cevaplar.

5 Dashboard'un Bugünkü Kullanım Alanı

İlk iterasyonun çekirdek sorgu değerlendirmesi aşağıdaki alanlar etrafında şekillenmektedir:

- Normalize SQL ve queryid
- Çağrı sayısı
- Toplam ve ortalama execution süresi

- Fiziksel shared block okuması
- Temp block kullanımı
- Mevcut dönem ile önceki eş dönem karşılaştırması
- Regresyon yüzdesi
- Impact Score ve öncelik sınıfı
- Temel tablo sağlık sinyalleri

5.1 Bugünkü kullanımın güçlü tarafı

Bu seçim, ilk iterasyonun temel amacıyla uyumludur:

```
Which queries create the highest total impact,
and which ones should be inspected first?
```

Sistem yalnızca ortalama süresi yüksek sorgulara bakmak yerine toplam yük, çağrı sıklığı, fiziksel okuma, temp kullanımı ve regresyonu birlikte değerlendirmektedir. Bu, ilk ürün sürümü için doğru bir önceliklendirme çekirdeğidir.

5.2 Bugünkü kullanımın sınırı

Mevcut dashboard, ham veri envanterinin aşağıdaki bölümlerinden yeterince yararlanmamaktadır:

- Süre değişkenliği ve kararlılık
- Cache hit ve gerçek I/O zamanı
- Satır başına verimlilik
- Temp spill miktarı ve süresi
- WAL ve yazma etkisi
- Veri kalitesi ve sayaç reset bilgileri
- Saatlik yük davranışı ve yeni ortaya çıkan ağır sorgular
- Paralellik ve JIT açıklamaları

6 Kullanılmayan Yüksek Değerli Analizler

6.1 Performans kararlılığı

Kullanılacak alanlar:

```
min_exec_time
max_exec_time
mean_exec_time
stddev_exec_time
```

Ortalaması düşük görünen bir sorgu zaman zaman çok yavaş çalışabilir. Bu nedenle sadece mean değerine bakmak kullanıcı deneyimini tam yansıtmaz.

6.1.1 Önerilen türetilmiş metrikler

- **Max/Mean oranı:** uç değer riskini gösterir
- **Variation coefficient:** $\text{stddev} / \text{mean}$

- **Stability grade:** düşük, orta, yüksek değişkenlik

Yorum sınırı

Standart sapma gerçek p95 veya p99 değildir. Dağılımın şekli bilinmediğinden percentile olarak sunulmamalıdır.

6.2 Satır verimliliği

Kullanılacak alanlar:

```
rows
calls
total_exec_time
shared_blks_hit
shared_blks_read
```

Önerilen metrikler:

$$\begin{aligned} \text{Satır / çağrı} &= \frac{\text{rows}}{\text{calls}} \\ \text{Süre / satır} &= \frac{\text{total_exec_time}}{\max(\text{rows}, 1)} \\ \text{Blok / satır} &= \frac{\text{shared_blks_hit} + \text{shared_blks_read}}{\max(\text{rows}, 1)} \end{aligned}$$

Bu metrikler, az satır sonucu için çok yüksek miktarda veri işleyen sorguları inceleme adayı olarak işaretleyebilir.

6.3 Cache ve I/O bağımlılığı

$$\text{Cache Hit Oranı} = \frac{\text{shared_blks_hit}}{\text{shared_blks_hit} + \text{shared_blks_read}}$$

`track_io_timing` açıksa execution süresinin I/O bölümüne yakın bir gösterge üretilebilir:

$$\text{Read Time Payı} = \frac{\text{shared_blk_read_time}}{\text{total_exec_time}}$$

Yorum sınırı

Düşük PostgreSQL buffer hit oranı, her zaman yavaş fiziksel disk anlamına gelmez. Veri işletim sistemi page cache'inde bulunabilir. Gösterge "I/O bağımlılığı sinyali" olarak sunulmalı, kesin disk teşhisi olarak sunulmamalıdır.

6.4 Temp spill analizi

Kullanılacak alanlar:

```
temp_blks_read
temp_blks_written
temp_blk_read_time
temp_blk_write_time
```

Önerilen çıktılar:

- Toplam temp okuma/yazma boyutu

- Çağrı başına temp boyutu
- Temp I/O süresi
- Temp kullanan sorguların toplam DB yükündeki payı
- “Sort/hash belleğe sığmıyor olabilir” bulgusu

Execution plan olmadan kesin neden veya doğrudan `work_mem` önerisi verilmemelidir.

6.5 WAL ve yazma etkisi

Kullanılacak alanlar:

```
shared_blks_dirtied
shared_blks_written
wal_records
wal_fpi
wal_bytes
wal_buffers_full
```

Önerilen çıktılar:

- Toplam WAL üretimi
- Çağrı başına WAL
- Yazma etkisi sınıfı
- Replication ve disk yazma baskısı inceleme sinyali
- Okuma optimizasyonu ile yazma maliyeti arasındaki denge için temel veri

Bu alan, ileride index önerilerinin risk skorunda kullanılmalıdır. Yeni index okumayı hızlandırırken yazma ve WAL maliyetini artırabilir.

6.6 Planning maliyeti

Planning takibi açılırsa aşağıdaki analizler üretilebilir:

- Toplam planning süresi
- Çağrı başına planning süresi
- Planning / execution oranı
- Planning süresi değişkenliği

Etkinleştirme kararı

`pg_stat_statements.track_planning` varsayılan olarak açık kabul edilmemelidir. Özellikle yüksek eşzamanlı sistemlerde etkinleştirilmeden önce yük testi ve ölçüm yapılmalıdır.

6.7 Parallel worker ve JIT görünürlüğü

Bu alanlar ana dashboard'da değil, sorgu detayının ileri seviye bölümünde gösterilmelidir:

- Planlanan ve başlatılan parallel worker farkı
- JIT generation, optimization ve emission süreleri
- JIT süresinin toplam execution içindeki yaklaşık payı

Bunlar ileri seviye DBA açıklaması sağlar; genel kullanıcı ekranını kalabalıklaştırmamalıdır.

7 PoWA Geçmişinden Çıkarılabilecek Ek Değer

PoWA remote mode; metrik snapshot'larını Collector aracılığıyla kaynak sunuculardan alır ve ayrı repository üzerinde saklar. Repository, retention ve farklı veri kaynağı sürümleri arasında tutarlı görünüm sağlar.

Asıl ürün değeri yalnızca mevcut kolonları göstermekten değil, zaman serisi davranışını anlamlandırmaktan gelir.

7.1 Top movers

- Son 24 saatte en hızlı kötüleşen sorgular
- Toplam yük payı en hızlı yükselen sorgular
- Fiziksel okuma payı en hızlı artan sorgular
- Temp kullanımı yeni başlayan sorgular

7.2 Yeni ağır sorgular

Bir `queryid` önceki dönemde hiç görünmüyor veya ihmal edilebilir düzeydeyken mevcut dönemde yüksek yüke çıkıyorsa “yeni ağır sorgu” olarak işaretlenebilir.

7.3 Saatlik ve günlük yük haritası

- Günün hangi saatlerinde yük yükseliyor?
- Belirli batch veya raporlama pencerelerinde hangi sorgular baskın?
- Hafta içi ve hafta sonu davranışı farklı mı?

7.4 Deployment öncesi ve sonrası

Deployment zamanı kullanıcı tarafından işaretlenirse aşağıdaki karşılaştırmalar üretilebilir:

- Ortalama ve toplam execution süresi değişimi
- Çağrı hacmi değişimi
- Fiziksel okuma ve temp kullanım değişimi
- Yeni query pattern'leri

8 Veri Kullanım Matrisi

Veri alanı	Bugünkü kullanım	Potansiyel değer	Önerilen çıktı
calls	Yüksek	Yüksek	Çağrı sıklığı, dönemsel hacim değişimi
total_exec_time	Yüksek	Yüksek	Toplam DB etkisi, yük payı
mean_exec_time	Yüksek	Yüksek	Çağrı başına gecikme
min/max/stddev_exec_time	Düşük	Yüksek	Kararlılık ve outlier riski
rows	Düşük	Orta/Yüksek	Satır başına verimlilik

Veri alanı	Bugünkü kullanım	Potansiyel değer	Önerilen çıktı
shared_blks_read	Orta/Yüksek	Yüksek	Fiziksel okuma baskısı
shared_blks_hit	Düşük	Yüksek	Cache hit ve cache bağımlılığı
I/O zamanları	Düşük	Yüksek	I/O ağırlıklı sorgu sınıflandırması
Temp block alanları	Orta	Yüksek	Spill miktarı ve temp maliyeti
WAL alanları	Düşük	Yüksek	Yazma etkisi ve replication baskısı
Planning alanları	Yok/opsiyonel	Orta	Planning ağırlıklı sorgular
JIT alanları	Yok	Düşük/Orta	İleri seviye detay açıklaması
Parallel worker alanları	Yok	Düşük/Orta	Paralellik gerçekleşme farkı
stats_reset	Düşük	Yüksek	Veri kapsamı ve karşılaştırma güveni
dealloc	Düşük	Yüksek	Takip kapasitesi/veri kaybı uyarısı
PoWA zaman serisi	Orta	Çok yüksek	Top movers, yeni sorgu, heatmap, deployment karşılaştırması

9 Önerilen İterasyon 1.1

9.1 P0 - Ürün değerini doğrudan artıran işler

1. Sorgu kararlılığı

- Min, max ve standart sapma görünümü
- Max/mean ve variation coefficient
- “Dalgalı performans” finding’i

2. Cache ve I/O görünümü

- Cache hit oranı
- Çağrı başına physical read
- I/O timing açıksa read/write süresi

3. Temp spill görünümü

- Toplam ve çağrı başına temp boyutu
- Temp I/O süresi
- Spill severity sınıfı

4. Top movers

- En hızlı kötüleşen sorgular

- Yük payı en çok artan sorgular
- Yeni ağır sorgular

5. Veri kalitesi kartı

- Son stats reset
- Dealloc artışı
- Snapshot boşluğu
- Veri güveni: yüksek/orta/düşük

9.2 P1 - Analiz derinliğini artıran işler

1. Satır verimliliği metrikleri
2. WAL ve yazma etkisi sekmesi
3. Saatlik/günlük yük heatmap'i
4. Deployment işaretleme ve öncesi/sonrası karşılaştırması
5. Tablo/index sayaçlarının dönemsel delta'ları

9.3 P2 - İleri seviye detaylar

1. Planning istatistiklerini kontrollü etkinleştirme
2. Paralel worker gerçekleşme analizi
3. JIT maliyet görünümü
4. Kullanıcı tanımlı eşik ve finding kuralları

10 Önerilen Dashboard Tasarımı

10.1 Sorgu listesi sade kalmalıdır

Ana sorgu listesinde aşağıdaki alanlar yeterlidir:

Impact	Priority	SQL	Calls	Total Time
Mean Time	Load %	Regression	Findings	

Ana tabloya tüm ham metrikleri eklemek okunabilirliği azaltır.

10.2 Sorgu detay sekmeleri

Sekme	İçerik
Özet	Impact Score, priority, yük payı, calls, total/mean time, temel findings
Gecikme ve Kararlılık	Min, max, mean, stddev, max/mean, dönemsel trend
I/O ve Cache	Hit/read, cache hit, read/write timing, çağrı başına physical read
Temp Kullanımı	Temp read/write boyutu, süre, çağrı başına değer, severity
Yazma ve WAL	Dirtied/written block, WAL records, FPI, bytes ve çağrı başına WAL

Sekme	İçerik
Geçmiş	1/7/30 gün trendi, önceki dönem, top mover durumu
Veri Kalitesi	Stats reset, dealloc, snapshot boşlukları ve güven seviyesi
İleri Seviye	Planning, JIT ve parallel worker alanları

11 Önerilen Finding Kuralları

Kurallar kesin teşhis değil, inceleme önceliği üretmelidir.

Finding	Örnek koşul	Kullanıcı mesajı
Dalgali çalışma süresi	Yeterli calls ve yüksek stddev/mean	Sorgu çalışma süresi kararsız; parametre veya cache etkisi olabilir.
Yüksek outlier riski	Max/mean belirlenen eşğin üzerinde	Bazı çalışmalar ortalamadan çok daha yavaş.
I/O ağırlıklı	Read time / exec time yüksek	Execution süresinin önemli bölümü block okuma beklemesine gidiyor.
Düşük cache hit	Yeterli block hacminde hit oranı düşük	Sorgu cache dışı veri okumaya daha bağımlı görünüyor.
Temp spill	Temp written ve çağrı başına temp yüksek	Sorgu yoğun geçici disk kullanıyor; plan incelenmeli.
Yüksek WAL etkisi	WAL bytes veya call başına WAL yüksek	Sorgu önemli yazma/replication yükü oluşturuyor.
Yeni ağır sorgu	Önceki dönemde yok, mevcut dönemde yüksek load	Bu sorgu yeni ortaya çıktı ve DB yükünde anlamlı paya ulaştı.
Veri güveni düşük	Reset yakın veya dealloc artıyor	Rapor dönemi eksik olabilir; sayaç kapsamını kontrol edin.

Eşik tasarımı

Sabit evrensel eşikler yerine, minimum veri hacmi + mutlak eşik + ortam içi percentile birlikte kullanılmalıdır. Örneğin yalnızca 3 çağrısı olan bir sorgu için standart sapma veya cache oranı güçlü bir karar sinyali sayılmamalıdır.

12 Veri Kalitesi ve Yanlış Pozitif Riskleri

12.1 Sayaç resetleri

PostgreSQL veya extension sayaçları sıfırlandığında, reset öncesi ve sonrası kümülatif değerler doğrudan karşılaştırılmamalıdır. Her tarihsel seri bir reset sınırı taşımamalıdır.

12.2 Deallocation

`pg_stat_statements.max` yetersizse az kullanılan kayıtlar çıkarılır. Bu durum düşük frekanslı sorguların tarihsel görünürlüğünü azaltır ve normalize SQL metninin kalitesini etkileyebilir.

12.3 Düşük örnek sayısı

Min/max/stddev alanları düşük çağrı sayısında yanıltıcı olabilir. Kararlılık finding'leri için minimum calls eşiği uygulanmalıdır.

12.4 Cache yorumları

PostgreSQL shared buffer miss, her zaman gerçek disk erişimi değildir; veri işletim sisteminin cache'inde olabilir. Dashboard dili buna uygun olmalıdır.

12.5 Satır alanının anlamı

rows, SELECT için dönen; DML için etkilenen satır sayısını temsil eder. Farklı sorgu türleri aynı verimlilik formülüyle doğrudan karşılaştırılmamalıdır.

12.6 Query ID kararlılığı

queryid, aynı platform ve minor sürümlerde yararlı bir kimliktir; ancak PostgreSQL major sürümleri arasında kalıcı evrensel kimlik olarak kabul edilmemelidir.

13 Mevcut Veriyle Üretilmeyecek Sonuçlar

Kesin sınırlar

Aşağıdaki sonuçlar mevcut `pg_stat_statements` + PoWA verisiyle güvenilir şekilde üretilemez:

- Gerçek p95 ve p99 execution süresi
- Execution plan node'ları ve node bazlı maliyet
- Hangi WHERE/JOIN predicate'inin yük oluşturduğu
- Kesin CPU süresi
- Wait event dağılımının geçmişi
- Gerçek parametre değerleri
- Güvenilir otomatik CREATE INDEX önerisi
- Güvenilir DROP INDEX önerisi
- Gereksiz JOIN kararı
- Otomatik SQL rewrite

Bu özellikler için çağrı bazlı histogram/telemetry, execution plan, `pg_qualstats`, HypoPG, wait sampling veya ek analiz katmanları gerekir.

14 Uygulama Planı

14.1 Aşama 1 - Veri uygunluk kontrolü

1. Kaynak PostgreSQL major sürümünü tespit et.
2. `pg_stat_statements` görünümündeki mevcut kolonları envanterle.
3. `track_io_timing` durumunu kontrol et.
4. Planning takibinin durumunu kontrol et; hemen açma.

5. PoWA adapter ve repository tablolarında hangi alanların tarihsel olarak taşındığını doğrula.
6. Reset ve dealloc bilgilerinin snapshot kapsamını doğrula.

14.2 Aşama 2 - Repository ve SQL katmanı

1. Yeni alanlar için dönemsel delta hesapları ekle.
2. Sıfıra bölme ve reset sınırı kontrollerini ekle.
3. Kararlılık, cache, temp, WAL ve veri kalitesi fonksiyonlarını oluştur.
4. Top movers ve yeni ağır sorgular için karşılaştırma view'ları ekle.
5. Yüksek hacimde performans için uygun index ve rollup ihtiyacını ölç.

14.3 Aşama 3 - FastAPI

1. Query detail response modelini sekmelere uygun genişlet.
2. Dashboard overview için data quality ve top movers endpoint'leri ekle.
3. Alan bulunmadığında null/unsupported durumunu açık döndür.
4. PostgreSQL sürümüne bağlı kolonları feature capability olarak raporla.
5. Export davranışını yeni metriklerle güncelle.

14.4 Aşama 4 - React dashboard

1. Ana sorgu listesini sade tut.
2. Query detail için yeni sekmeleri ekle.
3. Veri güveni uyarılarını görünür yap.
4. Top movers ve new heavy queries kartlarını ekle.
5. Finding metinlerinde kesin teşhis yerine inceleme dili kullan.

14.5 Aşama 5 - Test ve doğrulama

1. Sayaç reset senaryosu
2. Sıfır calls/rows/blok senaryoları
3. Düşük örnek sayısı
4. Dealloc artışı
5. Temp ve WAL üreten örnek workload
6. I/O timing açık/kapalı davranışı
7. PostgreSQL sürüm uyumluluğu
8. 90 günlük repository yük testi

15 Kabul Kriterleri

İterasyon 1.1 aşağıdaki koşullar sağlandığında tamamlanmış kabul edilmelidir:

- Sorgu detayında kararlılık, cache/I/O, temp, WAL ve geçmiş sekmeleri çalışıyor.
- Desteklenmeyen alanlar sıfır gibi gösterilmiyor; “veri mevcut değil” olarak işaretleniyor.
- Reset sınırını aşan hatalı delta hesaplanmıyor.
- Veri kalitesi kartı stats reset, dealloc ve snapshot boşluklarını gösteriyor.
- Top movers en hızlı kötüleşen sorguları zaman aralığına göre sıralıyor.
- Yeni ağır sorgu tespiti önceki eş dönemle karşılaştırılıyor.
- Düşük örnekli sorgularda kararlılık bulgusu bastırılıyor.
- Ana sorgu listesi mevcut sadeliğini koruyor.
- 90 günlük test verisinde endpoint süreleri kabul edilen sınırdan kalıyor.
- Finding metinleri kesin index veya yapılandırma önerisi üretmiyor.

16 Beklenen Ürün Etkisi

Geliştirme	Kullanıcıya etkisi	Teknik etkisi
Kararlılık analizi	Ortalama değer gizlediği ani yavaşlamalar görünür olur	Parametre/cache kaynaklı değişkenlik için araştırma sinyali
Cache ve I/O	Yavaşlığın veri erişimiyle ilişkisi daha iyi anlaşılır	Plan olmadan güvenli I/O sınıflandırması
Temp spill	Disk tabanlı sort/hash şüpheleri görünür olur	İkinci iterasyondaki plan analizi için aday üretir
WAL etkisi	Yazma sorgularının sistem maliyeti görünür olur	Index önerisi risk değerlendirmesine veri sağlar
Top movers	En pahalı değil, en hızlı kötüleşen sorgular yakalanır	Regresyon tespitinin operasyonel değeri artar
Veri kalitesi	Kullanıcı raporun ne kadar güvenilir olduğunu bilir	Yanlış delta ve eksik query seti riski azalır

17 Sonuç

İlk iterasyon, ürünün temel amacını doğru biçimde yerine getirmektedir: sorguları normalize etmek, tarihsel yükü toplamak, dönemsel karşılaştırma yapmak ve inceleme önceliği üretmek. Ancak mevcut veri kaynakları, bugünkü dashboard’un kullandığından daha fazla anlamlı bilgi sağlamaktadır.

Önerilen İterasyon 1.1 ile mimari değiştirilmeden ve çoğunlukla yeni extension eklenmeden aşağıdaki sorular cevaplanabilir:

- Sorgu yalnızca pahalı mı, yoksa aynı zamanda kararsız mı?
- Sürenin önemli bölümü veri okuma beklemesine mi gidiyor?
- Sorgu geçici disk kullanımına ne kadar yük bindiriyor?

- Sorgunun yazma ve WAL etkisi nedir?
- Hangi sorgular yeni ortaya çıktı veya en hızlı kötüleşti?
- Gösterilen raporun veri kalitesi ne kadar yüksek?

Nihai öneri

İlk iterasyon kapatılabilir; fakat otomatik optimizasyon önerilerine geçmeden önce İterasyon 1.1 uygulanmalıdır. Böylece ikinci iterasyonun index ve SQL önerileri daha iyi adaylar, daha güçlü geçmiş bağlamı ve daha doğru risk sinyalleri üzerine kurulacaktır.

18 Kaynaklar

1. PostgreSQL 18 Documentation, *pg_stat_statements - track statistics of SQL planning and execution*:
<https://www.postgresql.org/docs/current/pgstatstatements.html>
2. PostgreSQL 18 Documentation, *The Cumulative Statistics System*:
<https://www.postgresql.org/docs/current/monitoring-stats.html>
3. PoWA 5.0 Documentation, *PoWA Architecture*:
<https://powa.readthedocs.io/en/latest/architecture.html>
4. PoWA 5.0 Documentation, *Remote Setup*:
https://powa.readthedocs.io/en/latest/remote_setup.html
5. SQL Dashboard project repository:
<https://github.com/McanKul/sql-dashboard>